Adversarial Search

Asst.Prof.Rapeeporn Chamchong, PhD

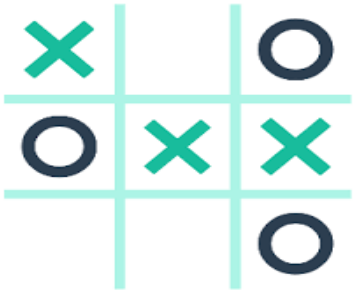
Source: 1) Soke slides from Zainab Yousuf,
2) ความรู้เบื้องต้นทางปัญญาประดิษฐ์ โดย ผู้ช่วยศาสตราจารย์ ดร. ชูพันธุ์ รัตนโภคา

Content

- Adversarial Environment
- Adversarial Search
  - Minimax Procedure
  - Alpha-beta pruning

Adversarial Search (การค้นหาแบบมีคู่ต่อสู้)

- Adversarial search is a type of search in which two or more agents search the search space with contrary goals.
- Such scenarios usually occur in games, where two opponents, also called adversaries, look for opposite goals.
- For example,

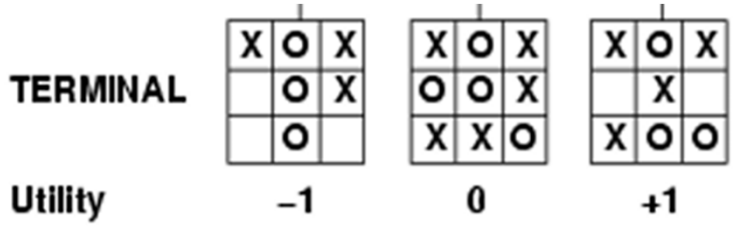


Tic-tac-toe game

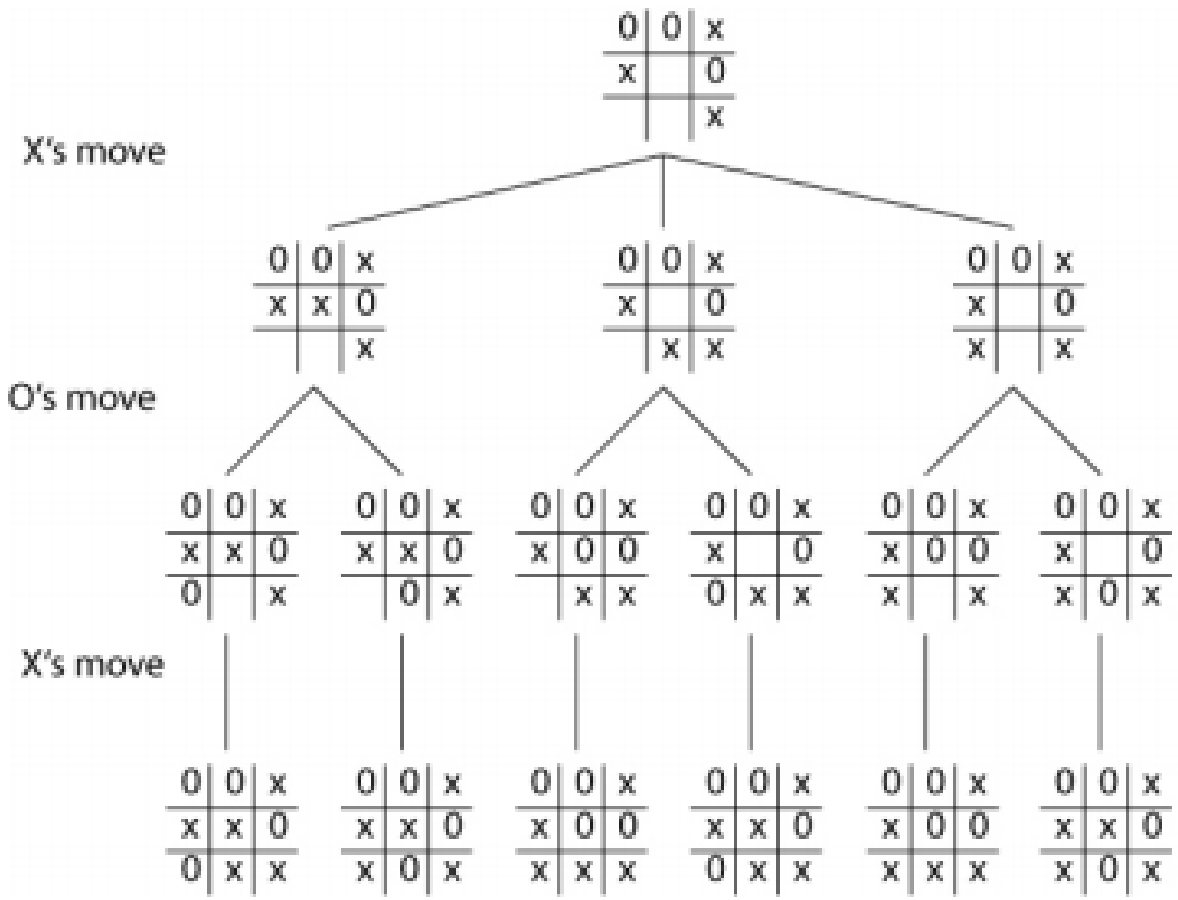
Chess

Basic Components to Define

- The basic components to define in Adversarial Search Techniques are as follows:
  - **Initial state**: Defines the initial state.
  - **Successor Function**: Defines the set of all possible operations.
  - **Terminal Test**: Defines the termination state. The termination state is called the terminal state.
  - **Utility Function**: A function that assigns a value to the terminal state as a number indicating the result of the game, such as win (+1), lose (-1), draw (0). If some games have complex outcome measures, a value may be used.



Tic-tac-toe as adversarial search



- In Tic-tac-toe, one player tries to complete a line with crosses, while the other aims for a line with zeros, resulting in different goals for both.
- Further, if player one puts a cross in any box, player two will intelligently try to make a move aimed at minimizing player one's chances of winning. This means player two will attempt to prevent player one from completing a line of crosses while also working to complete his own line of zeros.

Look-ahead strategy in multiplayer games

- In adversarial searches, one player anticipates the opponent's moves by intelligently evaluating the impact of their own move on the overall game configuration.
  - What would be the impact of his/her move?
  - What options will be open to the opponent once he has made this move?
  - What will the game situation be after a few turns?
- This Intelligent behavior is referred to as a look-ahead strategy.
- In AI, this look-ahead strategy is implemented using the Minimax Procedure and Alpha-Beta Pruning.

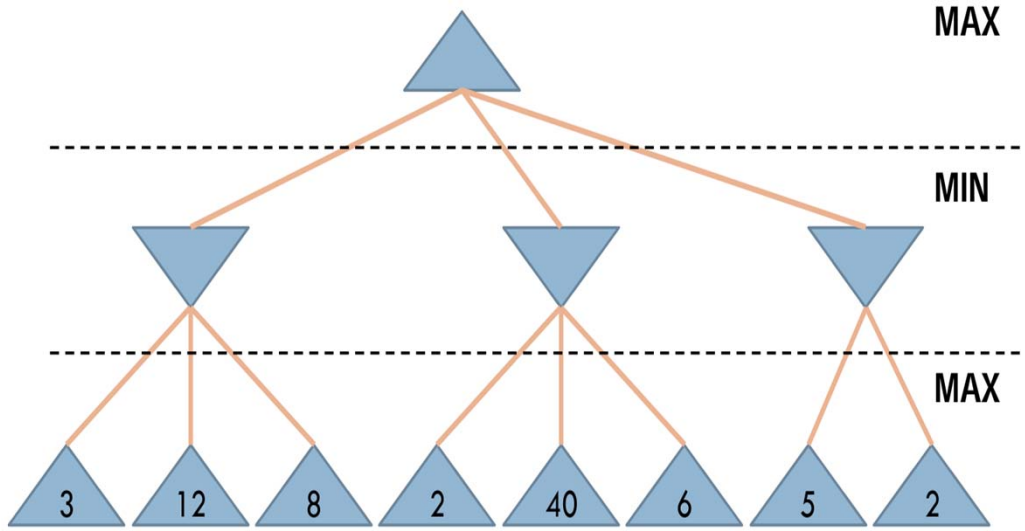
Minimax Procedure

- Use a DFS principle by selecting values from leaf nodes to determine the parent node's value, continuing until the root node receives a value.
- The game under consideration is usually a two-player game.
  - One player is designated as "MAX" (representing us). MAX's objective is to score the highest possible points or find a way to win.
  - The other player is "MIN," whose objective is to try to make MAX's score the lowest possible, or to try to make MAX lose.

Max Node Min Node

Example Minimax

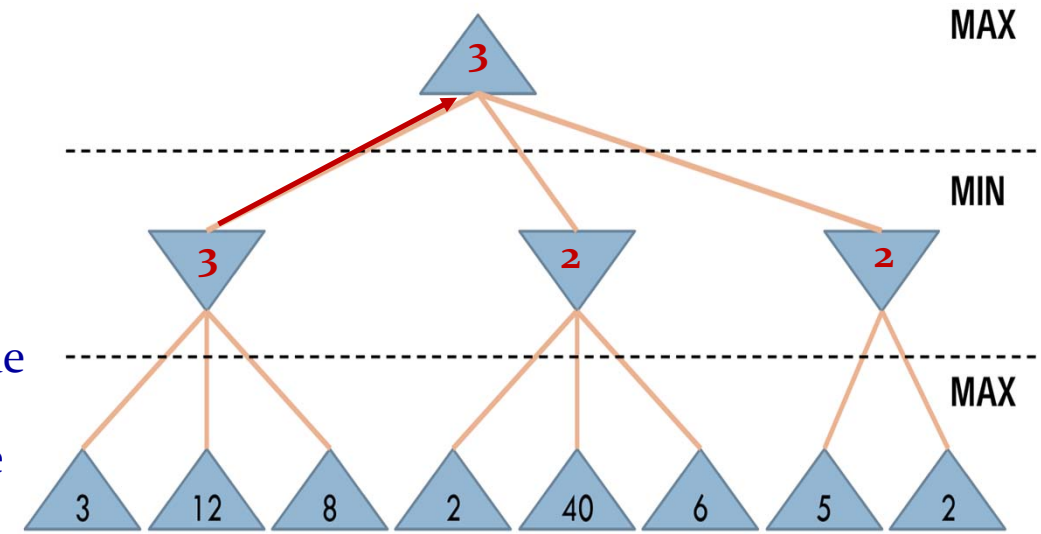
- The path the player should choose will be selected the best number
  - MAX node will choose the highest number
  - MIN node will choose the lowest number.



- In this example, it's a tree structure with 3 branching factors. The leaf nodes have various values defined by the game developer for each end state.
- Each node has a MinimaxValue, which can be calculated using:

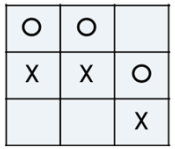
$$\text{MinimaxValue(n)}=\begin{cases} Utility(n); & if\ n\ is\ terminal\ state \\ max_{s\in Successor(n)}\ MinimaxValue(s); & if\ n\ is\ max \\ min_{s\in Successor(n)}\ MinimaxValue(s); & if\ n\ is\ min \end{cases}$$

Minimax Procedure



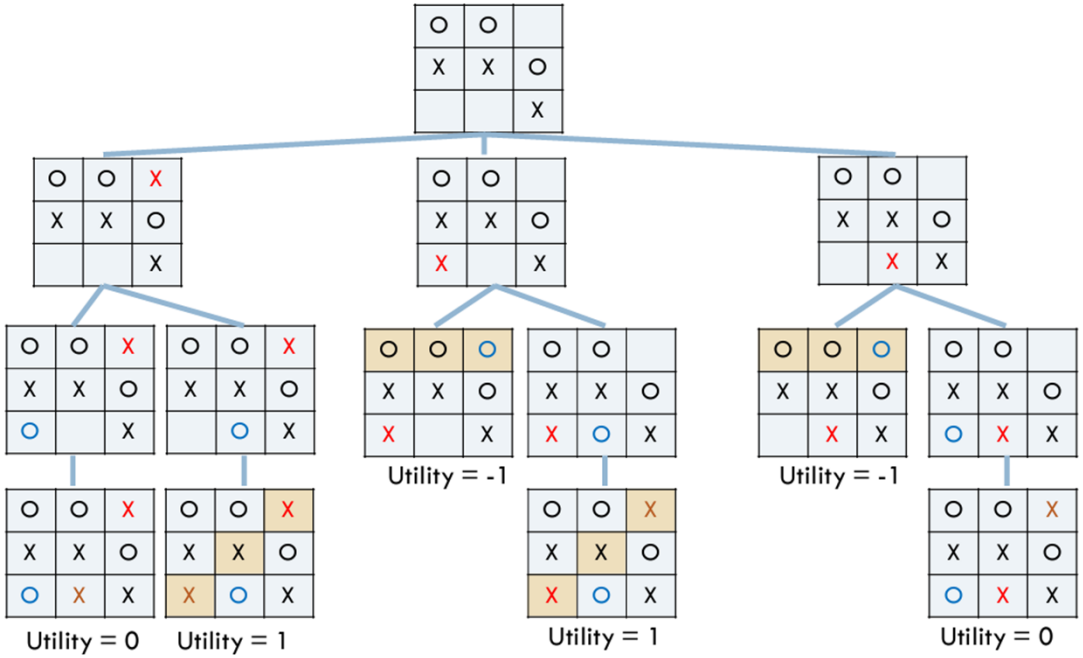
- Rules to move the heuristic upward level-by-level from the left subtree.
  - The leftmost MIN node will have a MinimaxValue which will be the smallest value from its child nodes: 3, 12, and 8. Therefore, the MinimaxValue of the MIN node is min{3, 12, 8} = 3.
  - The middle MIN node will have a MinimaxValue which will be the smallest value from its child nodes: 2, 40, and 6. Therefore, the MinimaxValue of the MIN node is min{2, 40, 6} = 2.
  - The rightmost MIN node will have a MinimaxValue which will be the smallest value from its child nodes: 5, and 2. Therefore, the MinimaxValue of the MIN node is min{5, 2} = 2.
  - When the MinimaxValue of all MIN nodes is obtained, the MAX load, which is the root of the tree, is considered. The MinimaxValue of MAX is calculated from max{3, 2, 2} = 3.
- The best path for the player to reach the target load is to transition to the left child state.

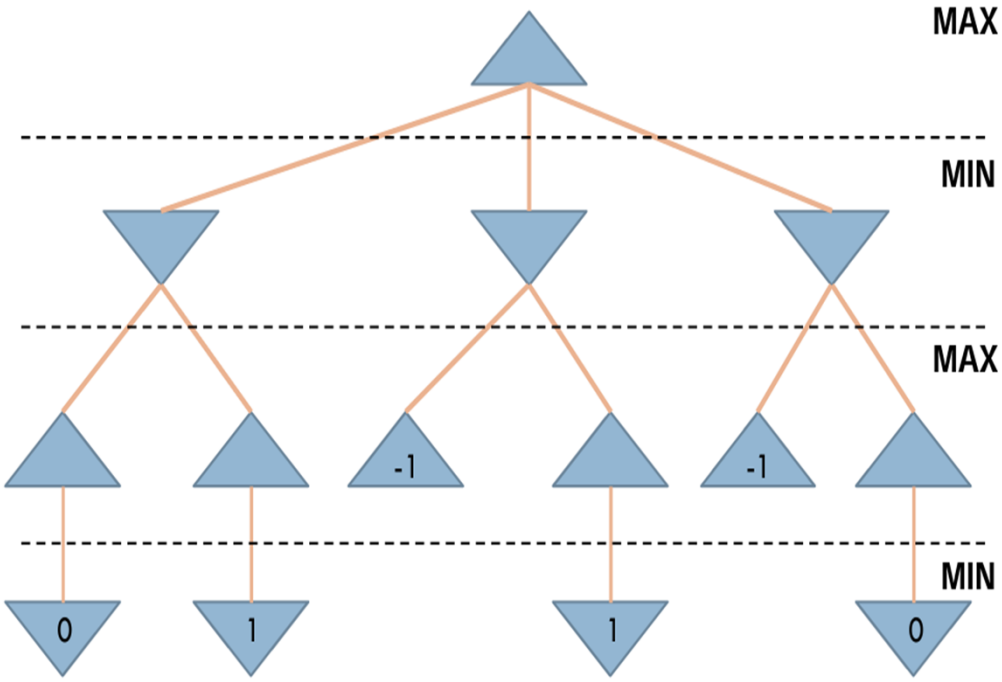
Example Minimax – Tic-Tac-Toe



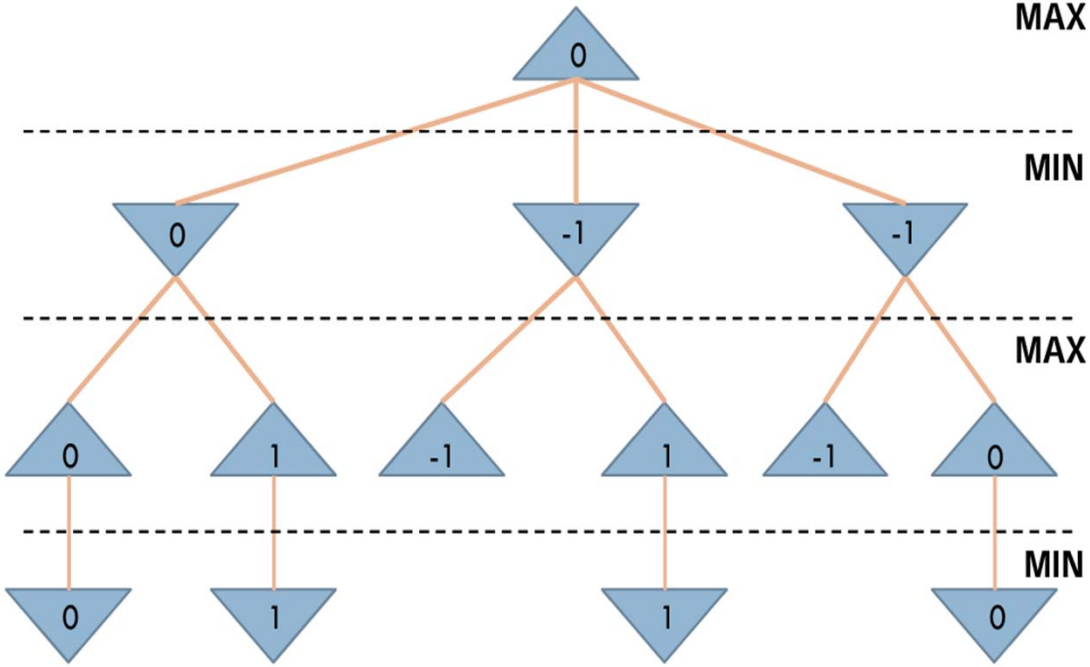
Current state

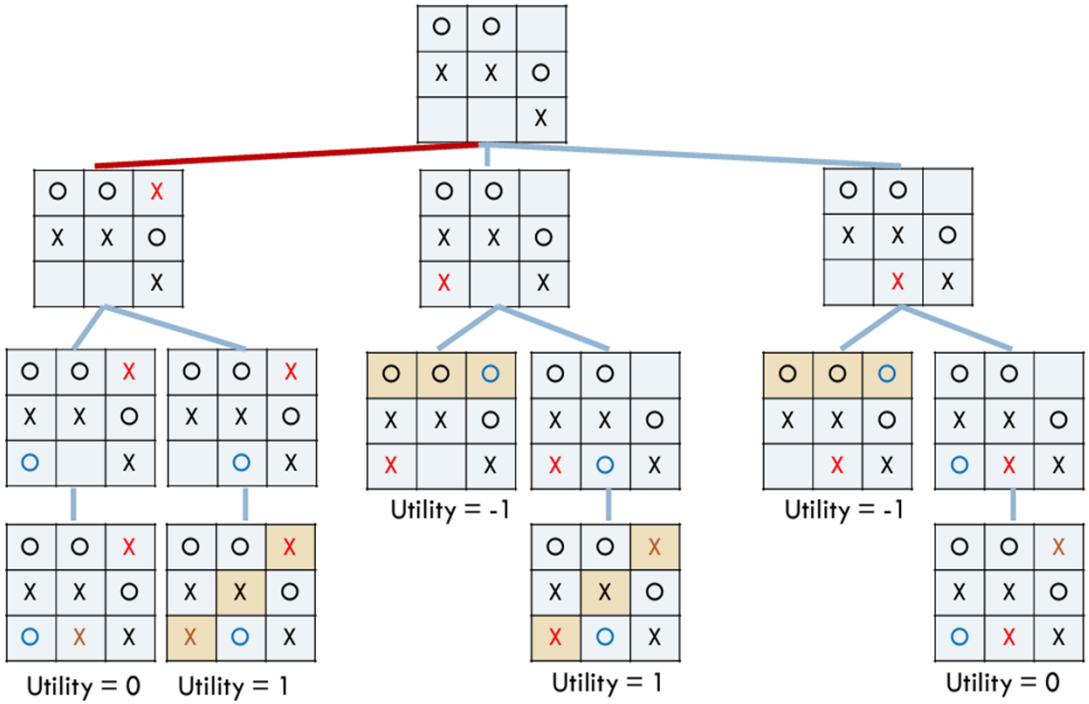
Unity=1, if X wins.
Unity=-1 if X loses.
Unity=0 if X draws





State space with the unity function to terminal state


MAX
MIN
MAX
MIN
0
0
-1
-1
0
1
-1
1
-1
0
0
1
1
0


Utility = -1
Utility = -1
Utility = 0
Utility = 1
Utility = 1
Utility = 0

Algorithm Minimax

```
FUNCTION Minimax(node, depth, isMaximizingPlayer)
    IF depth == 0 OR node is terminal THEN                    // Check the terminal state
        RETURN utility value of node
    END IF
    IF isMaximizingPlayer THEN                                // This is for the player MAX (AI)
        bestValue = -INFINITY
        FOR EACH child of node DO
            value = Minimax(child, depth - 1, FALSE)
            bestValue = MAX(bestValue, value)
        END FOR
        RETURN bestValue
    ELSE                                                      // if the opposite player MIN (คู่ต่อสู้)
        bestValue = +INFINITY
        FOR EACH child of node DO
            value = Minimax(child, depth - 1, TRUE)
            bestValue = MIN(bestValue, value)
        END FOR
        RETURN bestValue
    END IF
END FUNCTION
```

Alpha-Beta Pruning

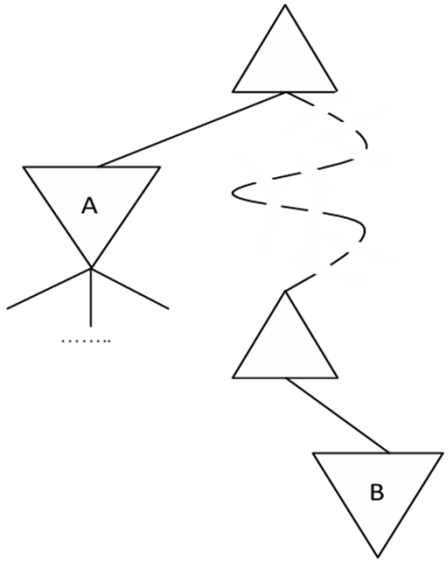
- Minimax search is a process that aims to find paths from every branch.
- If the game has a large exploration area, it will take more time for calculation and consume more memory because it has to store every path considered. Some paths may be unnecessary or not used.
- Therefore, an algorithm was developed to solve this problem, called "Alpha-Beta Pruning or Alpha-Beta Cutoff," which adds a technique to prune unnecessary branches or paths (Pruning/Cutoff) to save resources such as memory used and calculation time.

Alpha-Beta Pruning (cont.)

- Alpha-beta pruning is a procedure to avoid moving up all the heuristic values for all the branches of a tree by incorporating intelligent behavior.
- Alpha-beta search proceeds in a depth-first fashion.
- Two values, called alpha (α) and beta (β), are created during the search.
- The alpha value is associated with MAX nodes that can never decrease.
  - MAX node, $\alpha=6$. Then the value of MAX is not less than 6 from its immediate children (Min nodes).
- The beta value is associated with MIN nodes that can never increase.
  - MIN node, $\beta=6$. The value of MIN is not more than 6 from its immediate children (Max nodes).

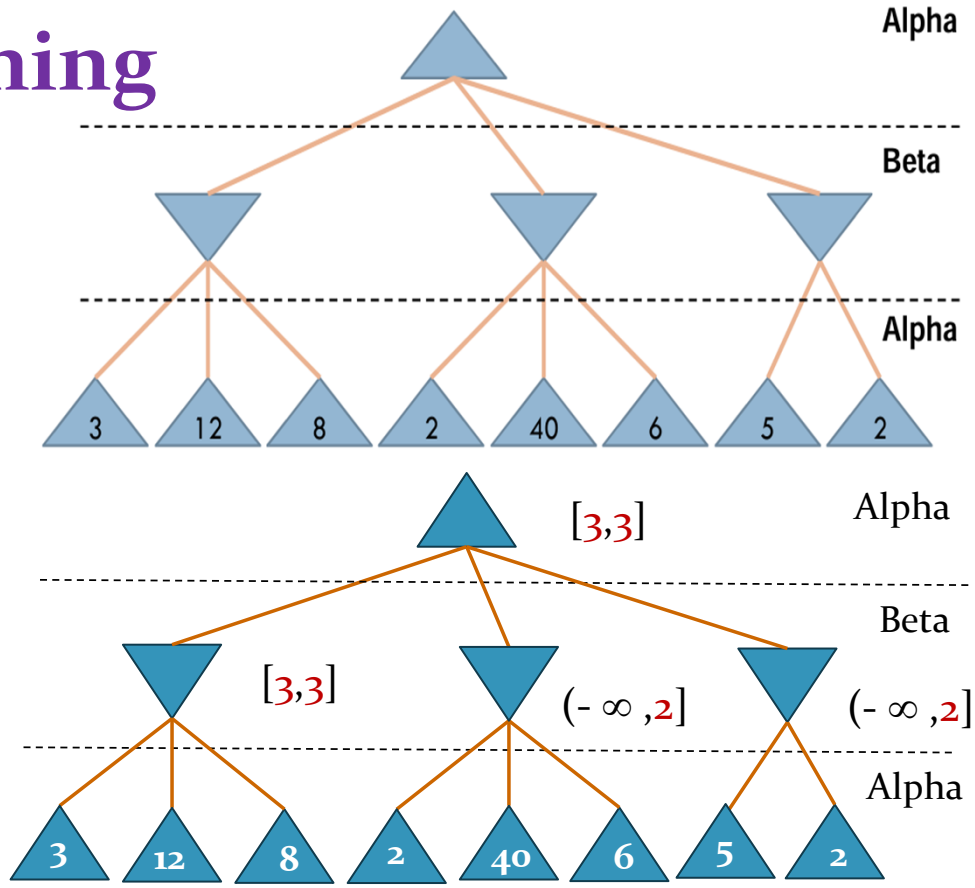
Alpha-Beta Pruning (Cont.)

- The method for considering pruning is to do so when it is proven that the Alpha range must be greater than or equal to Beta.
- Two rules for terminating search, based on alpha and beta values, are;
  - Search can be stopped below any MIN node having a beta value less than or equal to the alpha value of any of its MAX ancestors.
  - Search can be stopped below any MAX node having an alpha value greater than or equal to the beta value of any of its MIN node ancestors.



Example Alpha-Beta Pruning

- Step 1: Set initial value to root not in Alpha = (-∞,∞).
- Step 2: Explore the child states in depth on the left side, where the newly created nodes have node values of (-∞, ∞).
- Step 3: Explore the state of the left child first,starting from the leftmost node, which has a value of 3. The MIN node requires the smallest value; thus, the maximum value of the MIN node is 3, and the Beta range is (-∞,3].
- Step 4: Explore the status of the next child node, which has a node value of 12; therefore, the Beta value remains unchanged.
- Step 5: Consider the final value of this subtree, which is 8. It is greater than 3, so it is not usable. Therefore, Beta chooses the value 3 for the MIN node because it is the smallest. Thus, the range of Beta values is [3,3], and the range of Alpha values for the node above is [3, +∞), since the MAX node requires the largest value, at least 3.



Algorithm Alpha-Beta Pruning

```
FUNCTION Minimax(node, depth, alpha, beta, isMaximizingPlayer)
        IF depth == 0 OR node is terminal THEN
            RETURN value
        END IF
        IF isMaximizingPlayer THEN
            v = -INFINITY
            FOR EACH child of node DO
                v = MAX(v, Minimax(child, depth-1, alpha, beta, FALSE))
                alpha = MAX(alpha, v)
                IF beta <= alpha THEN
                    BREAK                                   // Pruning
                END IF
            RETURN v
        ELSE v = +INFINITY
            FOR EACH child of node DO
                v = MIN(v, Minimax(child, depth-1, alpha, beta, TRUE))
                beta = MIN(beta, v)
                IF beta <= alpha THEN
                    BREAK                                   // Pruning
                END IF
            RETURN v
        END IF
END FUNCTION
```

Advantage & Disadvantage

Feature	Minimax	Alpha-Beta Pruning
Exploration	Exhaustive (checks everything)	Selective (skips irrelevant paths)
Optimality	Guaranteed best move	Guaranteed best move (same as Minimax)
Time Complexity	$O(b^d)$	Best case: $O(b^{d/2})$
Search Depth	Limited (shallow)	Much deeper
Best Use Case	Educational / Tiny games	Real-world Game AI (Chess, etc.)